Resource Allocator - Interview Questions & Answers

Resource Allocator - 100 Interview Questions & Answers

Table of Contents

General Project Questions (1-20)

Q1: What is the Resource Allocator project?

Answer: Resource Allocator is a health activity scheduling system that transforms HealthSpan AI recommendations into personalized, actionable schedules. It intelligently schedules activities (fitness, nutrition, medication, therapy, consultations) while respecting constraints like equipment availability, specialist schedules, client preferences, and travel plans.

Q2: What problem does this project solve?

Answer: It solves the challenge of converting abstract health recommendations into practical daily schedules. Healthcare providers often give recommendations, but patients struggle to fit them into their busy lives while respecting resource availability, work schedules, and other constraints.

Q3: What are the main features of this system?

- Priority-based activity scheduling

- Constraint management (equipment, specialists, client availability)

- Multi-format output (Text, HTML, iCal, JSON)

- Interactive web interface with visualizations

- Test data generation

- Backup activity handling

- Travel day support

Q4: What technologies did you use and why?

- Python/Flask: Backend for scheduling logic and API

- React/TypeScript: Modern, type-safe frontend

- JSON: Simple data storage

- Recharts: Data visualization

- Chosen for simplicity, maintainability, and rapid development

Q5: How many activities can the system handle?

Answer: The system generates 105+ activities by default and can handle hundreds more. It's tested with 536 activities over 2 weeks. The architecture can scale to 1000+ activities with optimization.

Q6: What output formats does the system generate?

Answer: Four formats:

1. Text: Terminal/console-friendly with colors

2. HTML: Visual browser-viewable calendar

3. iCalendar (.ics): Import to Google Calendar, Apple Calendar, Outlook

4. JSON: Programmatic access for APIs

Q7: How does the system handle conflicts?

Answer: The system uses a priority-based approach:

1. Tries preferred time slots first

2. Falls back to any available time

3. Uses backup activities if primary unavailable

4. Logs all conflicts for review

5. Critical activities (medications) always scheduled first

Q8: What is the priority system?

Answer: Activities are prioritized 1-100:

- 1-20: Critical (medications, essential consultations)

- 21-50: High (fitness routines, key nutrition)

- 51-80: Medium (therapy, wellness)

- 81-100: Low (optional supplements)

Q9: How does frequency scheduling work?

Answer: The system supports:

- Daily: Every day

- Twice Daily: Morning and evening

- Weekly: Once per week (prefers Monday/Thursday)

- Twice Weekly: Tuesday and Friday
- Three Times Weekly: Monday, Wednesday, Friday
- Monthly: Once per month (mid-month)
- As Needed: Not auto-scheduled

Q10: What constraints does the scheduler consider?

- Equipment availability (gym hours, equipment access)
- Specialist schedules (doctor availability)
- Allied health schedules (therapist availability)
- Client work hours (9am-5pm blocked)
- Client sleep/wake times
- Travel plans (only remote activities during travel)
- Preferred time slots

Q11: How long does schedule generation take?

- Data generation: 2-5 seconds (105 activities, 3 months)
- Schedule generation: 5-15 seconds (2 weeks, 536 activities)
- API response: < 100ms (cached data)
- Frontend load: < 2 seconds

Q12: What happens during travel days?

Answer: The system handles travel specially:

- Only remote-capable activities are scheduled
- Medications are always scheduled (critical)
- Some fitness activities adapted (hotel gym)
- Activities get travel-specific notes
- Regular activities requiring equipment/specialists are skipped

Q13: How are backup activities used?

Answer: If a primary activity can't be scheduled (equipment unavailable, time conflict), the system:

1. Checks for backup activities defined in the activity
2. Tries to schedule the backup instead
3. Marks it as a backup activity
4. Tracks which activities were backups in statistics

Q14: What data does the system generate?

Answer: The system generates:

- 105+ health activities across 5 categories

- 36 equipment items with 3-month availability

- 8 specialists with schedules

- 8 allied health professionals

- 3 travel plans

- Client schedule with work hours and preferences

Q15: How is the project structured?

- Backend: Python modules (models, scheduler, datagenerator, calendaroutput, api)

- Frontend: React app with TypeScript

```
    CORS(app) # Enables CORS for all routes
```

This allows React (localhost:3000) to call Flask API (localhost:5001).

Q33: What is the API design philosophy?

Answer: RESTful principles:

```
    parts = time_str.split(":") return int(parts[0]) * 60 + int(parts[1])
```

Q42: How does frequency scheduling work?

- Track weekly/monthly counts per activity

- Check frequency enum

- Determine if should schedule today

- Reset counters at week/month boundaries

Q43: What data structures do you use?

- defaultdict: For grouping by date

- List: For activities and schedules

- Dict: For lookups (equipment, specialists)

- Set: For travel dates

- Dataclasses: For models

Q44: How do you generate test data?

Answer: DataGenerator class:

- Templates for each activity type

- Random but realistic data

- 3-month availability schedules

- Consistent relationships (equipment IDs match)

Q45: How do you serialize/deserialize data?

- todict(): Convert models to dictionaries
- fromdict(): Create models from dictionaries
- json.dump/load: File I/O
- Type-safe conversions

Q46: What Python features do you use?

- Dataclasses: Clean data models
- Enums: Activity types, frequencies
- Type hints: Function signatures
- List comprehensions: Data processing
- Context managers: File handling
- Defaultdict: Grouping data

Q47: How do you handle edge cases?

- Empty activity lists
- No available time slots
- Travel days
- Missing equipment
- Overlapping constraints
- All handled with checks and fallbacks

Q48: What is the complexity of the scheduler?

- Time: O(n * d * s) where n=activities, d=days, s=slots
- Space: O(n + d) for storing schedules and indices
- Optimized with early exits and indexing

Q49: How do you optimize performance?

- Build availability indices upfront
- Sort activities once
- Early exit on conflicts
- Cache frequently accessed data
- Minimize file I/O

Q50: How do you handle errors in Python?

- Try-except blocks
- Specific exception types

- Error logging
- User-friendly error messages
- Graceful degradation

React & Frontend (51-65)

Q51: Why TypeScript over JavaScript?

- Type safety catches errors early
- Better IDE support
- Self-documenting code
- Easier refactoring
- Fewer runtime errors

Q52: Explain the component structure.

- App.tsx: Main container, state management
- ConfigPanel: Form inputs
- StatisticsDashboard: Charts and stats
- ScheduleViewer: Date-based schedule display
- DownloadPanel: File downloads

Q53: How do you manage API calls?

Provides readable date formatting.

Q57: How do you handle user input?

- Controlled components
- State updates on change
- Form validation
- Submit handlers
- Error display

Q58: What is the styling approach?

- CSS modules per component
- Responsive design (media queries)
- Modern gradients
- Color-coded activity types
- Consistent spacing

Q59: How do you handle errors in React?

- Error state in components

- Try-catch in async functions

- User-friendly error messages

- Error boundaries (could add)

- Console logging for debugging

Q60: How do you ensure responsive design?

- CSS Grid and Flexbox

- Media queries for mobile

- Flexible layouts

- Touch-friendly buttons

- Readable font sizes

Q61: What React hooks do you use?

- useState: Component state

- useEffect: API calls, side effects

- Could use: useMemo, useCallback for optimization

Q62: How do you prevent unnecessary re-renders?

Answer: Currently rely on React's default optimization. Could add:

- useMemo for expensive calculations

- useCallback for function references

- React.memo for component memoization

Q63: How do you handle API connection status?

- Health check on mount

```
CORS(app) # Allows all origins (dev) # Production: CORS(app,
origins=["https://domain.com"])
```

Q83: What is the API response format?

Answer: Consistent JSON:

```
"success": true/false, "data": {...}, "error": "message" (if failed) }
```

Q84: How do you handle errors in the API?

- Try-catch blocks

- Return JSON error responses

- HTTP status codes (400, 404, 500)

- Log errors for debugging

- User-friendly messages

Q85: How would you add authentication?

- JWT tokens

- Login endpoint

- Token validation middleware

- Protected routes

- Refresh tokens

Q86: How do you handle file downloads?

```
limiter = Limiter(app, key_func=get_remote_address) @limiter.limit("10 per minute")
```

Q88: How do you validate input?

- Check required fields

- Validate date formats

- Validate ranges (weeks 1-24)

- Type checking

- Return 400 for invalid input

Q89: How would you add caching?

- Redis for API responses

- Cache schedule results

- Cache statistics

- TTL for cache expiration

- Invalidate on updates

Q90: How would you monitor the API?

- Logging (structured logs)

- Metrics (response times, errors)

- Health check endpoint

- APM tools (New Relic, Datadog)

- Error tracking (Sentry)

Problem Solving & Scenarios (91-100)

Q91: How would you handle 10,000 activities?

- Database instead of JSON
- Batch processing
- Indexing for fast lookups
- Parallel processing
- Caching frequently accessed data
- Optimize algorithm (reduce iterations)

Q92: A user says their schedule has conflicts. How do you debug?

1. Check scheduling log file
2. Verify activity priorities
3. Check resource availability
4. Review constraint validation
5. Test with smaller dataset
6. Add more detailed logging

Q93: How would you add real-time updates?

- WebSockets (Socket.io)
- Server-sent events
- Polling (simple but inefficient)
- Push notifications
- Update frontend state on changes

Q94: How would you handle multiple users?

- User authentication
- User-specific data storage
- Multi-tenancy in database
- User preferences per user
- Access control
- Data isolation

Q95: How would you optimize for mobile?

- Responsive design (already done)
- Touch-friendly buttons
- Mobile-first CSS
- Progressive Web App (PWA)
- Native mobile app (React Native)

- Offline support

Q96: How would you add schedule editing?

- Edit endpoint in API
- Frontend edit form
- Validation before save
- Conflict checking
- Update statistics
- Re-generate outputs

Q97: How would you add notifications?

- Email notifications (SMTP)
- SMS (Twilio)
- Push notifications (Firebase)
- In-app notifications
- Scheduled reminders
- Integration with calendar apps

Q98: How would you handle time zones?

- Store all times in UTC
- Convert to user timezone in frontend
- Use timezone-aware datetime
- Handle daylight saving
- User timezone preference
- Display in local time

Q99: How would you add machine learning?

- Collect user feedback
- Learn preferred times
- Predict optimal scheduling
- Personalize recommendations
- Optimize slot selection
- Improve over time

Q100: How would you deploy this to production?

1. Backend:

- Deploy Flask to AWS/GCP/Azure

- Use Gunicorn/uWSGI
- Set up database (PostgreSQL)
- Environment variables for config
- Load balancer

2. Frontend: